Introduction to Structures

- Structures in C (also called structs) allow grouping of different data types under one name.
- Unlike an array, a structure can contain many different data types (int, float, char, etc.).
- Each variable in the structure is known as a member of the structure.
- They are essential for managing complex data in an organized way.

- Syntax:

```
struct StructureName {
  // Members
};
```

Structures (struct) in C

- Structures (also called structs) are a way to group several related variables into one place.
- Each variable in the structure is known as a member of the structure.
- Unlike an array, a structure can contain many different data types (int, float, char, etc.).

```
struct MyStructure {    // Structure declaration
  int myNum;            // Member (int variable)
  char myLetter;        // Member (char variable)
}; // End the structure with a semicolon
```

Create a Structure

- You can create a structure by using the `struct` keyword and declare each of its members inside curly braces:

Access the structure

- To access the structure, you must create a variable of it.
- Use the struct keyword inside the main() method, followed by the name of the structure and then the name of the structure variable.

- Example : Create a struct variable with the name "s1":

Cont..

```
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  struct myStructure s1;
  return 0;
}
```

Access Structure Members

```c
// Create a structure called myStructure
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  // Create a structure variable of myStructure called s1
  struct myStructure s1;

  // Assign values to members of s1
  s1.myNum = 13;
  s1.myLetter = 'B';

  // Print values
  printf("My number: %d\n", s1.myNum);
  printf("My letter: %c\n", s1.myLetter);

  return 0;
}
```

Access Structure Members

```
#include <stdio.h>

// Create a structure called myStructure
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  // Create a structure variable of myStructure called s1
  struct myStructure s1;

  // Assign values to members of s1
  s1.myNum = 13;
  s1.myLetter = 'B';

  // Print values
  printf("My number: %d\n", s1.myNum);
  printf("My letter: %c\n", s1.myLetter);

  return 0;
}
```

Cont..

- Once creating a structure, you can easily create multiple structure variables with different values, using just one structure:

```
// Create different struct variables
struct myStructure s1;
struct myStructure s2;

// Assign values to different struct variables
s1.myNum = 13;
s1.myLetter = 'B';

s2.myNum = 20;
s2.myLetter = 'C';
```

```
s1 number: 13
s1 letter: B
s2 number: 20
s2 letter: C
```

```c
#include <stdio.h>

struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  // Create different struct variables
  struct myStructure s1;
  struct myStructure s2;
  // Assign values to different struct
variables
  s1.myNum = 13;
  s1.myLetter = 'B';

  s2.myNum = 20;
  s2.myLetter = 'C';
  // Print values
  printf("s1 number: %d\n", s1.myNum);
  printf("s1 letter: %c\n", s1.myLetter);

  printf("s2 number: %d\n", s2.myNum);
  printf("s2 letter: %c\n", s2.myLetter);
  return 0;
}
```

Strings in Structures?

- you can't assign a value to an array like this

```
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];  // String
};

int main() {
  struct myStructure s1;

  // Trying to assign a value to the string
  s1.myString = "Some text";

  // Trying to print the value
  printf("My string: %s", s1.myString);

  return 0;
}
```

An error will occur:

```
prog.c:12:15: error: assignment to expression with array type
```

Cont…

- a solution for this! You can use the strcpy() function and assign the value to s1.myString, like this:

```c
#include <stdio.h>
#include <string.h>

struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];  // String
};

int main() {
  struct myStructure s1;

  // Assign a value to the string using the strcpy function
  strcpy(s1.myString, "Some text");

  // Print the value
  printf("My string: %s", s1.myString);

  return 0;
}
```

```
My string: Some text
```

Simpler Syntax

- You can also assign values to members of a structure variable at declaration time, in a single line.

- Just insert the values in a comma-separated list inside curly braces {}. Note that you don't have to use the strcpy() function for string values with this technique:

Example

```c
// Create a structure
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];
};

int main() {
  // Create a structure variable and assign values to it
  struct myStructure s1 = {13, 'B', "Some text"};

  // Print values
  printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

  return 0;
}
```

```
13 B Some text
```

Modify Values

- If you want to change/modify a value, you can use the dot syntax (.).
- And to modify a string value, the strcpy() function is useful again:

```
#include <stdio.h>
#include <string.h>
// Create a structure
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];
};

int main() {
  // Create a structure variable and assign values to it
  struct myStructure s1 = {13, 'B', "Some text"};

  // Modify values
  s1.myNum = 30;
  s1.myLetter = 'C';
  strcpy(s1.myString, "Something else");

  // Print values
  printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

  return 0;
}
```

```
30 C Something else
```

Example 1

```c
// Create a structure variable and assign values to it
struct myStructure s1 = {13, 'B', "Some text"};

// Create another structure variable
struct myStructure s2;

// Copy s1 values to s2
s2 = s1;

// Change s2 values
s2.myNum = 30;
s2.myLetter = 'C';
strcpy(s2.myString, "Something else");

// Print values
printf("%d %c %s\n", s1.myNum, s1.myLetter, s1.myString);
printf("%d %c %s\n", s2.myNum, s2.myLetter, s2.myString);
```

```
13 B Some text
30 C Something else
```

Example 2

```c
#include <stdio.h>

struct Car {
  char brand[50];
  char model[50];
  int year;
};

int main() {
  struct Car car1  = {"BMW", "X5", 1999};
  struct Car car2  = {"Ford", "Mustang", 1969};
  struct Car car3  = {"Toyota", "Corolla", 2011};

  printf("%s %s %d\n", car1.brand, car1.model, car1.year);
  printf("%s %s %d\n", car2.brand, car2.model, car2.year);
  printf("%s %s %d\n", car3.brand, car3.model, car3.year);

  return 0;
}
```

```
BMW X5 1999
Ford Mustang 1969
Toyota Corolla 2011
```

Passing Structures to Functions in C Language

Why Use Structures in Functions?

Passing structures to functions enhances modularity, making code easier to manage and reuse.

Structures help in encapsulating related data, allowing functions to operate on complex data types.

Ways to Pass Structures

Structures can be passed to functions in two ways:

1. By Value

   (a copy is passed)

2. By Reference

   (a pointer to the original structure is passed)

Passing Structure by Value

When a structure is passed by value, a copy of the structure is passed to the function. This means changes in the function don't affect the original structure.

Syntax:

```
void function(struct StructureName var)
```

Example of Passing Structure by Value

```
struct Student {
    char name[50];
    int age;
};

void display(struct Student s) {
    printf("%s is %d years old.\n",
s.name, s.age);
}

struct Student student1 = {"Alice", 20};
display(student1);
```

Passing Structure by Reference

When a structure is passed by reference, a pointer to the original structure is passed. Any changes made in the function affect the original structure.

Syntax:

```
void function(struct StructureName *var)
```

Example of Passing Structure by Reference

```
struct Student {
    char name[50];
    int age;
};

void updateAge(struct Student *s) {
    s->age += 1;
}

struct Student student1 = {"Alice",
20};
updateAge(&student1);  // Modifies
student1's age
```

Choosing Between By Value and By Reference

- Passing by value is safer when you don't want to modify the original structure, while passing by reference is useful for efficiency and when updates to the structure are needed.

Summary and Best Practices

1. Use structures to group related data.
2. Pass by value to protect the original data.
3. Use by reference for efficiency and when modifications are required.
4. Always initialize structures before passing to functions.